Code Explanation

Code Explanation: AWS Glue PySpark Job Test SuiteThis is a **comprehensive pytest-based test suite** for an AWS Glue PySpark ETL job. It validates all functional requirements using mocked AWS services.---

High-Level Structure

1. **Imports & Setup**: Load dependencies, configure test paths2. **Pytest Fixtures**: Reusable test data and mock objects3. **Test Cases**: Organized by functional requirement (FR)4. **Integration Tests**: End-to-end pipeline validation5. **Error Handling Tests**: Exception scenarios---

Step-by-Step Breakdown

1. Imports and Path Configuration

```
import pytestfrom datetime import datetimefrom unittest.mock import Mock, patch, MagicMockfrom decimal import Decimalfrom pyspark.sql import SparkSession
```

- **pytest**: Testing framework- **unittest.mock**: Mock AWS services (S3, Glue) to avoid real API calls- **PySpark**: DataFrame operations and schema definitions- **sys.path manipulation**: Adds `../main` directory to import the actual `job.py` module being tested---

2. Pytest Fixtures (Reusable Test Components)

A. SparkSession Fixture

```
@pytest.fixture(scope="session")def spark_session():
```

- Creates a **local Spark session** for testing- `scope="session"`: Shared across all tests (performance optimization)- Configured with 2 shuffle partitions for faster local execution#### **B. Sample Data Fixtures****Clean Customer Data:**

```
@pytest.fixturedef sample_customer_data(spark_session):
```

- Creates 5 valid customer records- Schema: `custid`, `name`, `emailid`, `region`- Used for testing normal data flow**Dirty Customer Data:**

```
@pytest.fixturedef sample_customer_data_with_nulls(spark_session):
```

- Contains NULL values, "Null" strings, empty strings, and duplicates- Used for testing data cleaning logic**Order Data Fixtures:**- Similar pattern for order data (clean and dirty versions)- Schema: `orderid`, `itemname`, `priceperunit`, `qty`, `date`#### **C. Mock AWS Services****Mock S3 Client:**

```
@pytest.fixturedef mock_s3_client(): with patch('boto3.client') as mock_client:
```

- Mocks boto3 S3 client to avoid real AWS calls- Simulates successful bucket access and object listing- Returns controlled responses for testing**Mock Glue Context:**

```
@pytest.fixturedef mock_glue_context():
```

- Mocks AWS Glue context for catalog operations---

3. Test Cases by Functional Requirement

FR-VALIDATE-001: S3 Path Validation (4 tests)**Test 1: Valid S3 Path**

```
def test_validate_s3_path_valid(mock_s3_client):
```

- Verifies function returns `True` for valid `s3://` paths- Confirms `head_bucket` API call was made**Test 2: Invalid Format**

```
def test_validate_s3_path_invalid_format():
```

- Tests rejection of non-S3 paths (e.g., `/local/path`)**Test 3: Bucket Not Found**

```
def test_validate_s3_path_bucket_not_found(mock_s3_client):
```

- Simulates AWS 404 error using `ClientError`- Verifies graceful handling**Test 4: Empty Prefix**

```
def test_validate_s3_path_no_objects(mock_s3_client):
```

- Tests behavior when S3 prefix contains no objects---#### **FR-INGEST-001/002: Data Ingestion (5 tests)****Test 1: Read Customer Data**

```
def test_read_customer_data(spark_session, sample_customer_data, tmp_path):
```

- Writes sample data to temporary location- Reads it back using `read_data_safe()`- Validates record count and schema**Test 2: Read Order Data**- Similar pattern for order data ingestion**Test 3: Invalid Path Handling**

```
def test_read_data_invalid_path(spark_session):
```

- Mocks failed path validation- Confirms function returns `None`**Test 4: Column Normalization**

```
def test_read_data_column_normalization(spark_session, tmp_path):
```

- Creates data with mixed-case column names (`CustId`, `Name`)- Verifies all columns are converted to lowercase---#### **FR-CLEAN-001/002: Data Cleaning (5 tests)****Test 1: Remove NULL Values**

```
def test_clean_data_remove_nulls(sample_customer_data_with_nulls):
```

- Verifies rows with NULL values are removed- Checks no NULL values remain in any column**Test 2: Remove "Null" Strings**

```
def test_clean_data_remove_null_strings(sample_customer_data_with_nulls):
```

- Tests removal of literal "Null" string values**Test 3: Remove Duplicates**

```
def test_clean_data_remove_duplicates(sample_customer_data_with_nulls):
```

- Groups by `custid` and verifies max count = 1- Confirms no duplicate records remain**Test 4: Remove Empty Strings**- Validates empty string removal**Test 5: Clean Order Data**- Applies cleaning logic to order dataset---#### **FR-SCD2-001: SCD Type 2 Implementation (4 tests)****Test 1: Column Addition**

```
def test_apply_scd_type2_columns(sample_customer_data):
```

- Verifies `isactive`, `startdate`, `enddate`, `opts` columns are added**Test 2: IsActive Default**

```
def test_apply_scd_type2_isactive_default(sample_customer_data):
```

- Confirms all records have `isactive = True` by default**Test 3: Timestamp Population**

```
def test_apply_scd_type2_timestamps(sample_customer_data):
```

- Validates `startdate` and `opts` have current timestamp- Confirms `enddate` is NULL for active records**Test 4: Data Types**

```
def test_apply_scd_type2_data_types(sample_customer_data):
```

- Verifies correct PySpark data types: - `isactive`: BooleanType - `startdate/enddate/opts`: TimestampType---#### **FR-SCD2-002: Hudi Operations (2 tests)****Test 1: Hudi Configuration**

```
def test_write_hudi_table_configuration(sample_customer_data, tmp_path):
```

- Mocks `write_data_safe()` call- Verifies Hudi-specific options are passed: - `hoodie.table.name` - `hoodie.datasource.write.operation = "upsert"`**Test 2: Record Key Configuration**

```
def test_write_hudi_table_record_key(sample_customer_data, tmp_path):
```

- Confirms `hoodie.datasource.write.recordkey.field = "custid"`---#### **FR-AGG-001: Aggregation Logic (3 tests)****Test 1: Basic Aggregation**

```
def test_calculate_customer_aggregate_spend(sample_customer_data,
sample_order_data):
```

- Verifies `totalamount` and `totalqty` columns are created**Test 2: Total Amount Calculation**

```
def test_calculate_aggregate_total_amount(spark_session):
```

- Creates simple test data with known values- Validates: `totalamount = priceperunit * qty` - Example: 100.0 * 2 = 200.0**Test 3: Grouping Validation**

```
def test_calculate_aggregate_grouping(sample_customer_data, sample_order_data):
```

- Confirms results are grouped by `orderid`, `itemname`, `date`---#### **FR-OUTPUT-001: Data Write Operations (3 tests)****Test 1: Parquet Write**

```
def test_write_data_safe_parquet(sample_order_data, tmp_path):
```

- Writes data in Parquet format- Validates successful write operation**Test 2: Partitioned Write**

```
def test_write_data_safe_with_partitioning(sample_order_data, tmp_path):
```

- Tests partitioning by `date` column**Test 3: Empty DataFrame Handling**

```
def test_write_data_safe_empty_dataframe(spark_session, tmp_path):
```

- Verifies function returns `False` for empty DataFrames---#### **FR-CATALOG-001: Glue Catalog Registration (2 tests)****Test 1: Table Registration**

```
def test_register_glue_catalog_table(sample_order_data):
```

- Mocks Glue client- Verifies table registration succeeds when database exists**Test 2: Database Creation**

```
def test_register_glue_catalog_create_database():
```

- Simulates `EntityNotFoundException`- Confirms database is created automatically---

4. Integration Tests (2 tests)

Test 1: Customer Pipeline

```
def test_end_to_end_customer_pipeline(spark_session, sample_customer_data,
tmp_path):
```

- **Step 1**: Clean data- **Step 2**: Apply SCD Type 2- **Step 3**: Write to Parquet- Validates complete data flow**Test 2: Order Pipeline**

```
def test_end_to_end_order_pipeline(...):
```

- **Step 1**: Clean customer and order data- **Step 2**: Calculate aggregates- **Step 3**: Write partitioned summary- Tests full aggregation workflow---

5. Error Handling Tests (2 tests)

Test 1: Clean Data Exception

```
def test_clean_data_exception_handling(spark_session):
```

- Passes `None` as DataFrame- Expects exception to be raised**Test 2: SCD Type 2 Exception**- Similar pattern for `apply_scd_type2()`---

Key Testing Patterns Used

1. **Mocking**: Avoids real AWS API calls using `unittest.mock`2. **Fixtures**: Reusable test data and configurations3. **Temporary Paths**: `tmp_path` fixture for file I/O tests4. **Assertions**: Validates counts, schemas, data types, and values5. **Markers**: `@pytest.mark.unit` and `@pytest.mark.integration` for test categorization---

Test Coverage Summary

- **33 total test functions**- **All functional requirements covered**- **Unit tests**: Individual function validation- **Integration tests**: End-to-end pipeline validation- **Error handling**: Exception scenariosThis test suite ensures the Glue job meets all requirements before deployment to production.